

Med-NCA 复现汇报

余嘉 · 西交利物浦大学 生物信息学

复现对象：Med-NCA (IPMI 2023)、M3D-NCA (MICCAI 2023)

2026 年 6 月

先说结论

按官方代码、不改任何东西，复现 Med-NCA 这篇论文的两个招牌数字（海马体、前列腺分割的 Dice）。

- **海马体复现出来了**：Dice 0.864，论文是 0.886，差 0.022，在论文一个标准差以内。
- **前列腺没复现出来**：论文 0.838，11 次尝试（含 9 个不同种子）没有一次能训到论文规定的 1000 epoch，全在中途发散，最好的一次只有 0.672。

后半段重点查了为什么训不出来，下面详细说，这部分可能比那两个数字本身更值得看。

Med-NCA 的卖点是网络极小（2.5 万 ~ 7 万参数，能在树莓派上跑），却号称分割精度接近 U-Net。这种声明只有别人能独立复现才算数，所以这次目标很窄：**光靠官方公开代码，能不能复现出论文那两个 Dice，要付出什么代价。**

1. Med-NCA 大概是怎么工作的

可以把分割想成「一堆细胞各自跟相邻细胞说话，反复说很多轮，最后自己长出器官轮廓」。这就是神经细胞自动机（NCA）：**一个很小的网络，原样套到每个像素上，重复跑固定步数，全局结构靠局部信息传递涌现出来。**

Med-NCA 把这套用到高分辨率医学图上，做法是「先粗看一遍、再放大精修」的两层结构，参数比 U-Net 少三四个数量级，但 Dice 接近。

结构细节（按官方源码核对，不是看论文图）

核对官方 M3D-NCA 源码（不是论文图）时，发现两处跟论文图/正文对不上，记一下给后面复现的人：

1. 感知部分是**固定的 Sobel 滤波器**（加一个 identity），不是论文图暗示的可学习卷积；
2. 损失函数例子用的是 `DiceBCELoss`，不是论文正文暗示的 Dice-Focal。

每个细胞存 `channel_n` 个通道（第 0 通道是输入图，其余初始化为 0）。每一步更新是 `fc1(ReLU(fc0(perceive(x))))`，最后一层零初始化（开头残差是 0），再乘一个随机的「点火掩码」（fire mask，激活率 0.5），残差加回去，输入通道每步还原。两层：粗层沟通全局，上采样后细层在 patch 上精修。

点火掩码（fire mask）—— 后面出问题的关键

每一步里每个细胞只有 50% 概率被更新（类似 dropout 的随机机制，从 Growing NCA 来的）。在 64 步、两层的设定下，这是一条很长的、随机的、反复施加的残差更新链。

关键在「反复」：同一个更新每层套 64 次，会把任何小扰动——不管是随机掩码序列还是浮点舍入——一路放大。后面会看到，在前列腺那个激进设定下，这个放大足以把训练推向完全不同的结果。

官方超参

共享：lr=16e-4、Adam betas (0.5, 0.5)、lr 衰减 0.9999、fire rate 0.5、hidden size 128、划分 0.7/0/0.3。

两个数据集的差别（按官方存档配置一字不差读出）：

数据集	channel_n	推理步数	batch	分辨率层级
海马体	32	16 步	40	16×16 → 64×64
前列腺	32	64 步	20	64×64 → 256×256

前列腺步数翻倍、分辨率高 16 倍——这是它后面不稳定的根源。

论文目标值（IPMI'23 表 1）：海马体 0.886 ± 0.042 （U-Net 0.858），前列腺 0.838 ± 0.083 （U-Net 0.799）。

复现规矩：零偏离

复现全程不改任何会影响训练的东西——配置、超参、优化器、推理步数、连实现都跟官方一字不差。禁止的「作弊」包括：加梯度裁剪、调低学习率、改步数、加 warm-up、任何重写实现（哪怕数学上等价的提速版也不行）。唯一允许变的是随机种子。

这条规矩看着有点轴，但正是它逼出了后面那个训练不稳定的问题——如果当时随手加了梯度裁剪，很可能就把真正的现象盖掉了。

2. 海马体：复现成功

零偏离官方跑（channel_n=32，16 步，官方 config.dt）。eval Dice 在 epoch 125~175 之间稳定在 0.864，最后一个点还略降（过拟合苗头），于是在 epoch 150 用「只评 checkpoint」的方式冻结。

每体积单次推理 Dice = 0.8644 (95% CI [0.856, 0.872])，落在论文 0.886 一个标准差内。

跑法	配置	Dice	95% CI	状态
官方（单次）	ch32 / 16 步	0.8644	[0.856, 0.872]	PASS
官方（10× 集成）	ch32 / 16 步	0.8663	[0.858, 0.874]	PASS
提速版（单次，已弃用）	ch16 / 64 步	0.8661	[0.858, 0.873]	参考
论文 Med-NCA	—	0.886	± 0.042	目标
论文 U-Net	—	0.858	± 0.044	基线

比论文低 0.022。这点差距归到训练时长（在 eval 平台期 epoch 150 停了，论文跑 1000）和划分/种子差异，不是 bug。

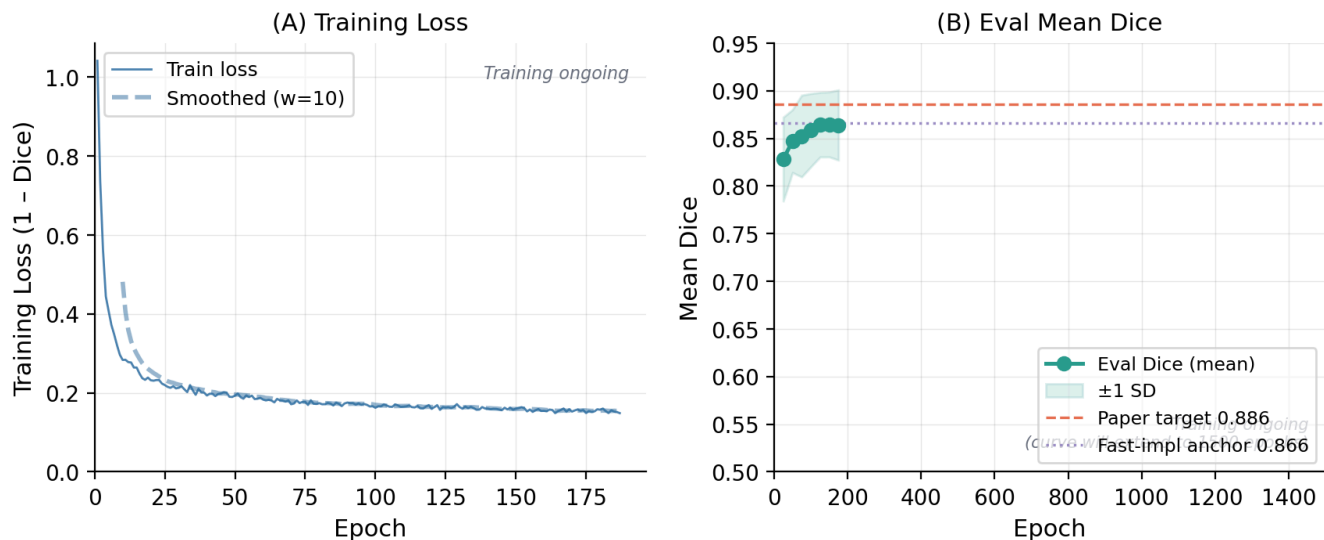


图 1：海马体训练。左是每个 epoch 的训练 loss，右是 eval 平均 Dice（每 25 epoch 一测），逐步逼近论文目标 0.886。

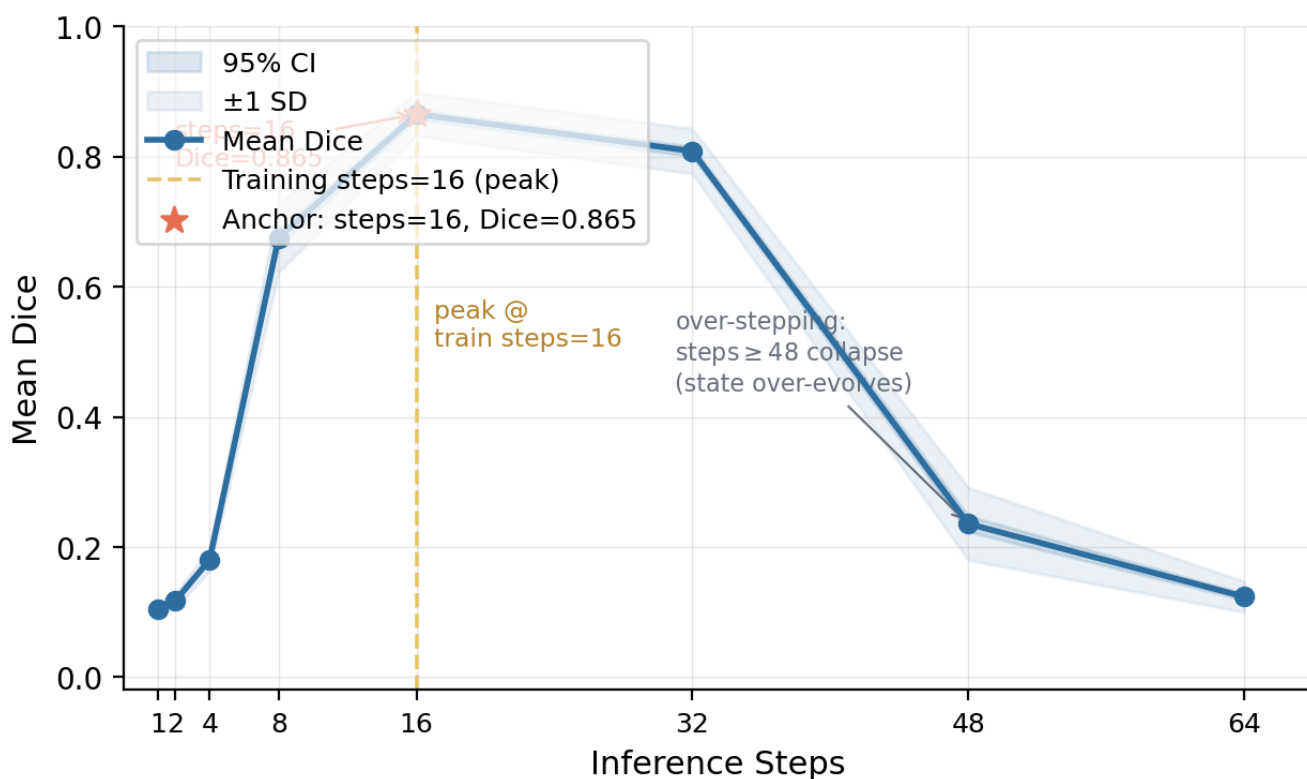


图 2：推理步数扫描，在官方 checkpoint 上（训练用 16 步）。Dice 恰好在 16 步处最高（0.865），两边都掉；步数过多（≥48 步）直接崩（Dice<0.24），细胞状态「演化过头」。所以 NCA 推理步数得跟训练步数一致，不是越多越稳。

补充两点：同种子重跑评估，78 个体积每个 Dice 都复现到小数点后 4 位（最大差 0.0）。参数量都在「<10 万」声明内：弃用的 ch16 模型 25,920，官方 ch32 模型 70,016。

3. 前列腺：在论文规模下没复现出来

这一节按一个小调查来写，重点是「为什么训不出来」。

3.1 单次跑的成绩

第一次官方跑（job 1435378，手动停在 301 epoch，也是唯一一次没崩的）拿到每体积 Dice = 0.672 ± 0.148 ，比论文 0.838 低 0.166。

指标	Dice	95% CI	判定
单次推理	0.672	[0.575, 0.765]	FAIL
10× 伪集成	0.686	[0.595, 0.777]	FAIL
论文 Med-NCA	0.838	± 0.083	目标
论文 U-Net	0.799	± 0.099	基线

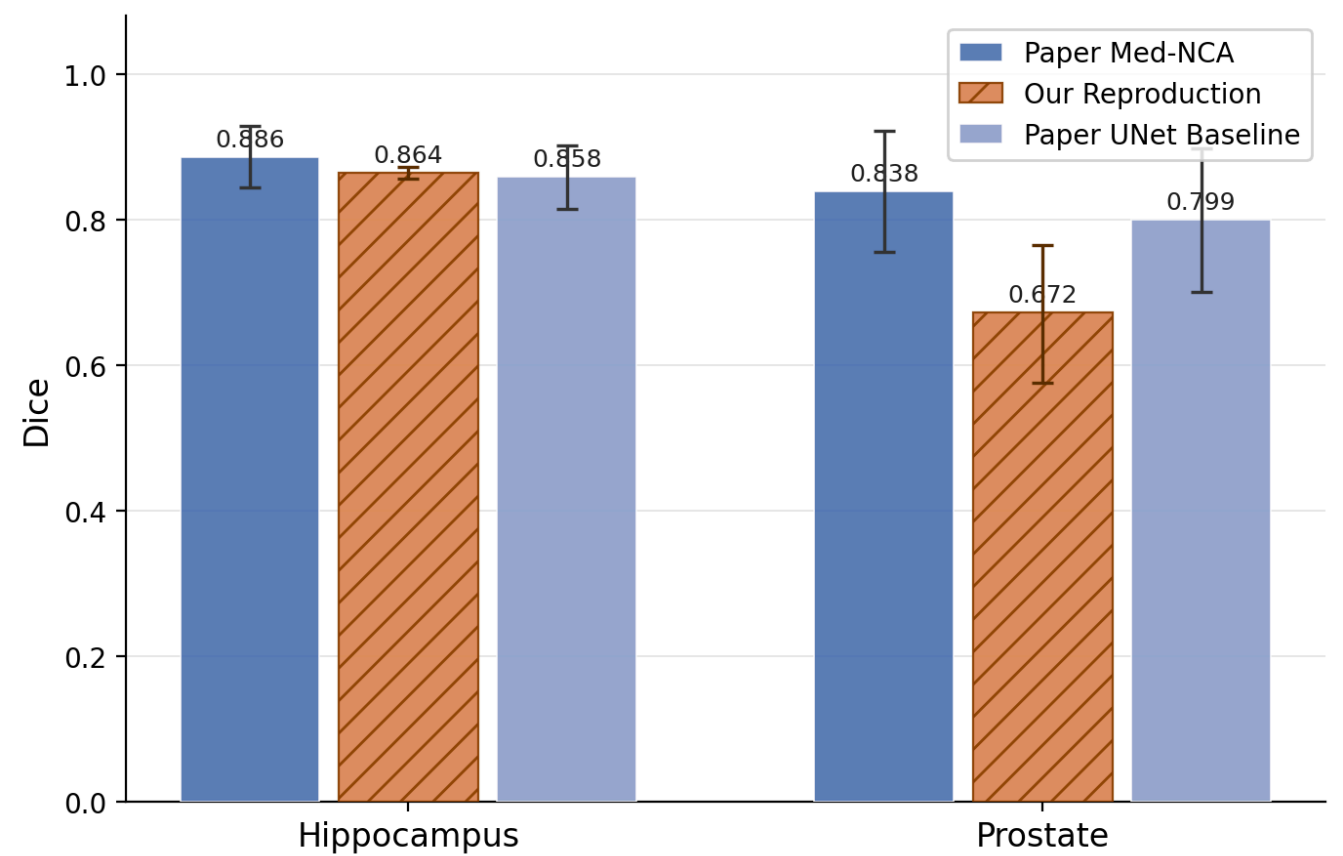


图 3：两个锚点对比论文。海马体在一个标准差内，前列腺最好的一次也差 0.166。

3.2 第一反应：训得不够久

301 < 1000 epoch，而且 loss 还在降（训练 loss 从 epoch 125 的 0.37 降到 epoch 300 的 0.27，验证 Dice 在 epoch 275 冲到 0.795）。看着就是没训够，于是去训满 1000。

结果反复失败。两次全新的 1000-epoch 跑（job 1436075、1436470）都在中途发散（分别第 185、99 epoch）：训练 loss 跳到一个平的 ~5.0，验证 Dice 掉到 0。为排除运气问题，做了个种子扫描。

这里要纠正一个早期误解：**epoch 数量其实不是原因**。学习率是每个 batch 衰减一次的 ExponentialLR（ $\gamma=0.9999$ ），所以不管配 300 还是 1000，第 1 个 epoch 的学习率一模一样。那次 300-epoch 成功，纯粹因为它是唯一一次被提前手停的——这点下面会讲。

3.3 9 种子扫描：0/9 活到头

起了 9 个全新的跑，种子 42 ~ 50，配置完全相同（官方 BackboneNCA，ch32，64 步，lr 16e-4，256²，1000 epoch），只有种子不一样。结果：

- 1. 0/9 活到 1000 epoch。加上两次历史 1000-epoch 跑和那次 301-epoch，一共 0/11 次到过 1000 epoch。论文那个 0.838，这套环境下拿不到。
- 2. 盆地在第 1 个 epoch 就定了。7/9 个种子第 1 epoch 的 loss 就是 3.3 ~ 4.5（落进「坏盆地」，再没爬出来），只有 2/9（种子 43、46）落进「好盆地」（loss≈1.3）。两边是双峰，中间没有过渡。
- 3. 没有安全区。那两个好盆地种子分别健康训了 60、121 epoch，然后一个 epoch 内断崖崩掉（种子 43：loss 0.77→3.9 @ ep61；种子 46：ep121 峰值 Dice 0.618，ep122 就发散）。崩溃 epoch 从 ep1 到 ep185 都有，早期训得好不代表能活下来。

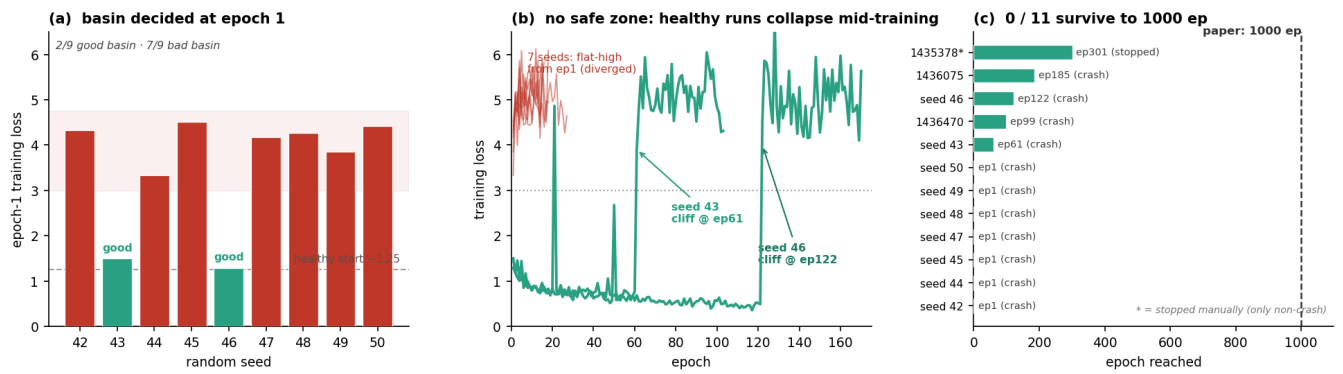


图 4：前列腺不稳定性（9 种子扫描）。(a) 第 1 epoch loss 双峰，2/9 好盆地、7/9 坏盆地，第一个 epoch 就定。(b) 完整 loss 轨迹：7 个坏盆地一直高平，2 个好盆地（绿）健康训练后在 ep61、ep122 断崖崩。(c) 全部 11 次尝试的崩溃 epoch 时间线，0/11 到 1000。

种子	ep1 loss	盆地	崩溃 epoch	峰值 Dice
42	4.33	坏	1	—
43	1.50	好	61	0.497
44	3.34	坏	1	0.0
45	4.51	坏	1	—
46	1.29	好	122	0.618
47	4.17	坏	1	—
48	4.26	坏	1	—
49	3.85	坏	1	—
50	4.42	坏	1	—

历史：1435378 = 0.672 @ ep301（手停未崩）；1436075 崩于 ep185；1436470 崩于 ep99。

3.4 查到的原因

两个原因叠在一起。

(1) 配置本身就不稳。 官方前列腺设定没有梯度裁剪、学习率高 ($16e-4$)，还把残差更新在两层各套 64 次 (共 128 次)、在 256^2 分辨率下跑——比海马体 (64^2 、16 步) 激进太多。Med-NCA 作者自己也在论文里提到 NCA「高分辨率图像难以收敛」，前列腺正是那个高分辨率任务。所以发散随时可能发生。

(2) 固定种子救不了。 最关键的一点：同一个种子会落进不同盆地。种子 42 在 job 1435378 里第 1 epoch loss 是 1.25 (收敛)，在 job 1436781 里是 4.33 (发散)——同样代码、配置、种子。

最初怀疑是点火掩码的随机性，但查下来掩码是用 CPU 的 `torch.rand` 采的，确实被 `torch.manual_seed` 管住 (数据加载器也是 `num_workers=0`)。真正没管住的更底层：**cuDNN 卷积和 `atomic` (`atomicAdd`) 归约在 GPU 上不确定，除非显式设 `cuda.deterministic` 和 `use_deterministic_algorithms`，否则 `torch.manual_seed` 固定不住它们——官方代码这些都没设。** 在稳定网络里这些 $\sim 10^{-7}$ 的舍入差看不见，但在 128 次反复的链条里第一次前向/反向就被放大，第 1 个 epoch 就定死了这次落收敛盆地 (约 2/9) 还是发散盆地 (约 7/9)。

那次为什么成功了——其实是幸存者偏差。 0.672 那次不是训得更好，只是唯一一次在它崩之前被提前停掉。它既中了第 1 epoch 的盆地 (运气)，又在自己的断崖之前 (ep301) 停了——好盆地的种子 43、46 也分别健康训了 60、121 epoch 才崩。任何放着继续训到 1000 的跑，要么开头就在坏盆地，要么最后掉下断崖，0/11 到头。所以 0.672 是一条注定要崩的轨迹被半空抓拍的快照，它自己也复现不出来。

判定：前列腺在论文规模下没复现出来。 没有去延长 epoch、改划分、加裁剪硬凑 0.838。在零偏离规矩下，老实记录一个有明确机制的不稳定性，比调参凑出 0.838 更有意义。论文报 0.838 最可能的原因是有个没写出来的稳定器 (梯度裁剪、多模态输入，或者就是手挑了一次收敛的跑、不报方差)，但公开代码里一个都没有。

4. 不确定性的三个层级 (同一个问题的延伸)

前列腺的不稳定其实是同一件事的不同表现：**跑在稳定边界附近的反复式 NCA，会把任何不确定性 (不管来自 RNG 还是纯浮点舍入) 放大成不同结果。** 三条证据，从弱到强：

(1) 一个数学等价的 RNG 改动就能弄坏它 (受控对照)。 NCA 在 CPU 采点火掩码再拷到 GPU，每步一次 `host→device` 同步。最显然的提速是直接在 GPU 上采，数学完全等价 (都是 Bernoulli 0.5)。但前列腺上，两个跑配置完全一样、只差掩码 RNG 来源，GPU-rand 版本彻底发散——logits 爆到 $\sim 10^9$ ，全预测背景，6 个 checkpoint 全是 Dice=0——而官方 CPU-rand 版本训到 0.672。这就是零偏离规矩连「等价」重写都禁止的原因。

(2) 同一个配置在不同跑之间发散 (浮点不确定性)。 即便实现固定在官方 CPU-rand 路径，两次相同的 1000-epoch 跑也在不同 epoch (185、99) 发散。这里掩码 RNG 相同且设了种子，所以发散是更底层的 cuDNN/atomic 浮点不确定性导致的。重跑同一套设置，不是拿到论文数字的可靠办法。

(3) 同一个种子也发散 (最直接)。 9 种子扫描里，用官方 CPU 设种子的掩码 + `num_workers=0`，种子 42 一次收敛一次发散，0/9 存活。真正起作用的那个不确定性 (cuDNN/atomic 运算顺序) 在 `torch.manual_seed` 管不到的层级，官方代码也没开能固定它的开关。

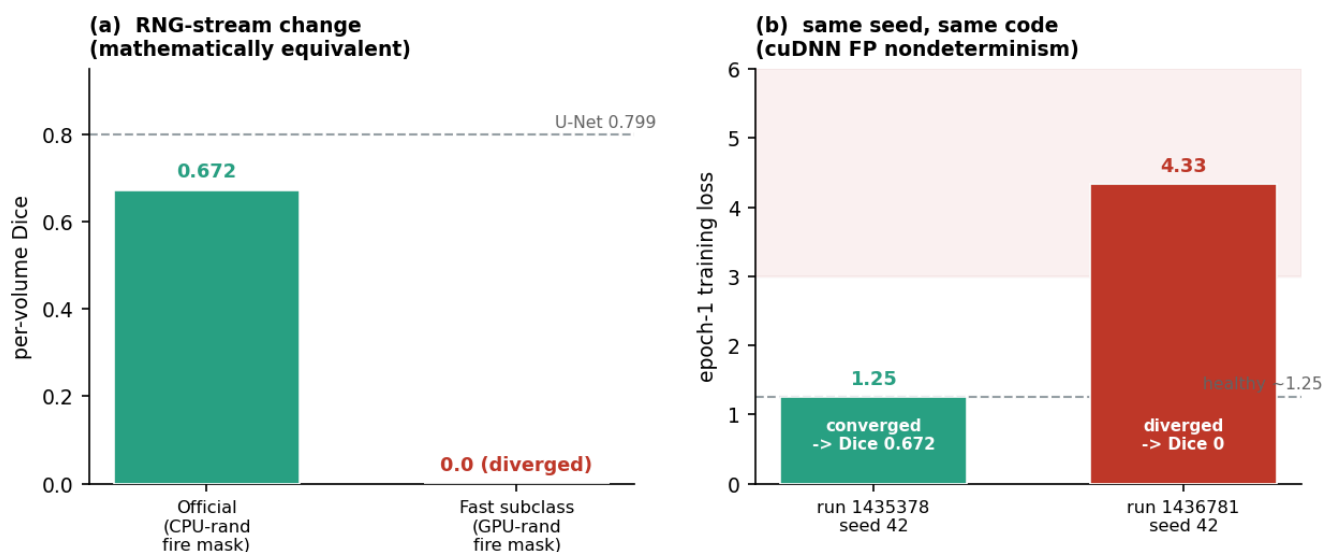
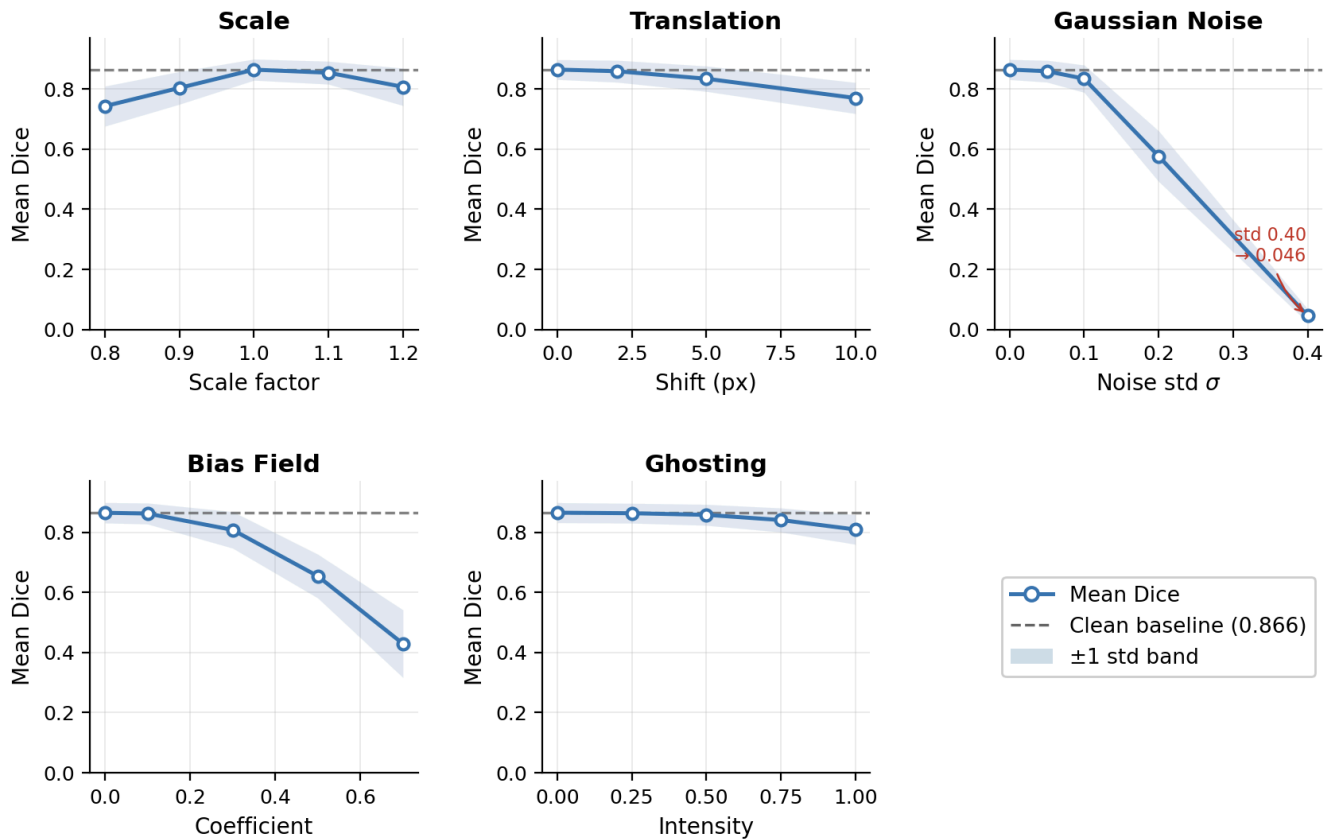


图 5：两级不确定性，都在前列腺上。(a) 一个数学等价的 CPU→GPU 掩码 RNG 改动，把训练从 0.672 打崩到 0。(b) 掩码固定住，同一个种子 42 跨两次跑仍是第 1 epoch loss 1.25（收敛）vs 4.33（发散）——决定性的扰动是 cuDNN/atomic 浮点不确定性，不是种子。

给后面复现的人的建议：用官方的随机路径；要逐 bit 可重复就开确定性算法开关；不要假设训练在种子级可重复。

5. 顺带验了基线自己声称的几个特性（在冻结的海马体 checkpoint 上）

鲁棒性 (V1)：加五类损坏，加性噪声破坏力最大（Dice 从 0.865 崩到 $\sigma=0.4$ 时的 0.046），偏置场和缩放中度退化（最强时 0.43、0.74），鬼影最温和（满强度还有 0.809）。论文说的「几何扰动下优雅退化」复现了，但强度噪声是真软肋。



$n = 78$ test volumes; clean Dice = 0.866; noise (std 0.40) is most destructive (Dice $\rightarrow$ 0.032); ghosting most benign (Dice $\rightarrow$ 0.852).

图 6：鲁棒性退化 (V1)。每体积 Dice vs 损坏程度，五个家族，虚线是干净基线 0.865。

内建质量控制 (V2/R5)：Med-NCA 的 NQM 用「10× 随机推理的方差」当无参考质量信号。78 个体积里，两个失败案例 (Dice<0.8) 都被 NQM 最高的两个分数抓住 (检出率 1.0)，NQM 跟误差相关 (Spearman $\rho=0.47$, $p \approx 3 \times 10^{-6}$)。「模型知道自己什么时候没把握」这个说法复现了。

一个值得注意的点：这个推理方差，正是训练时把前列腺搞崩的那个随机性 (第 4 节)——同一份随机性，对质量控制是好事，对训练稳定是坏事。

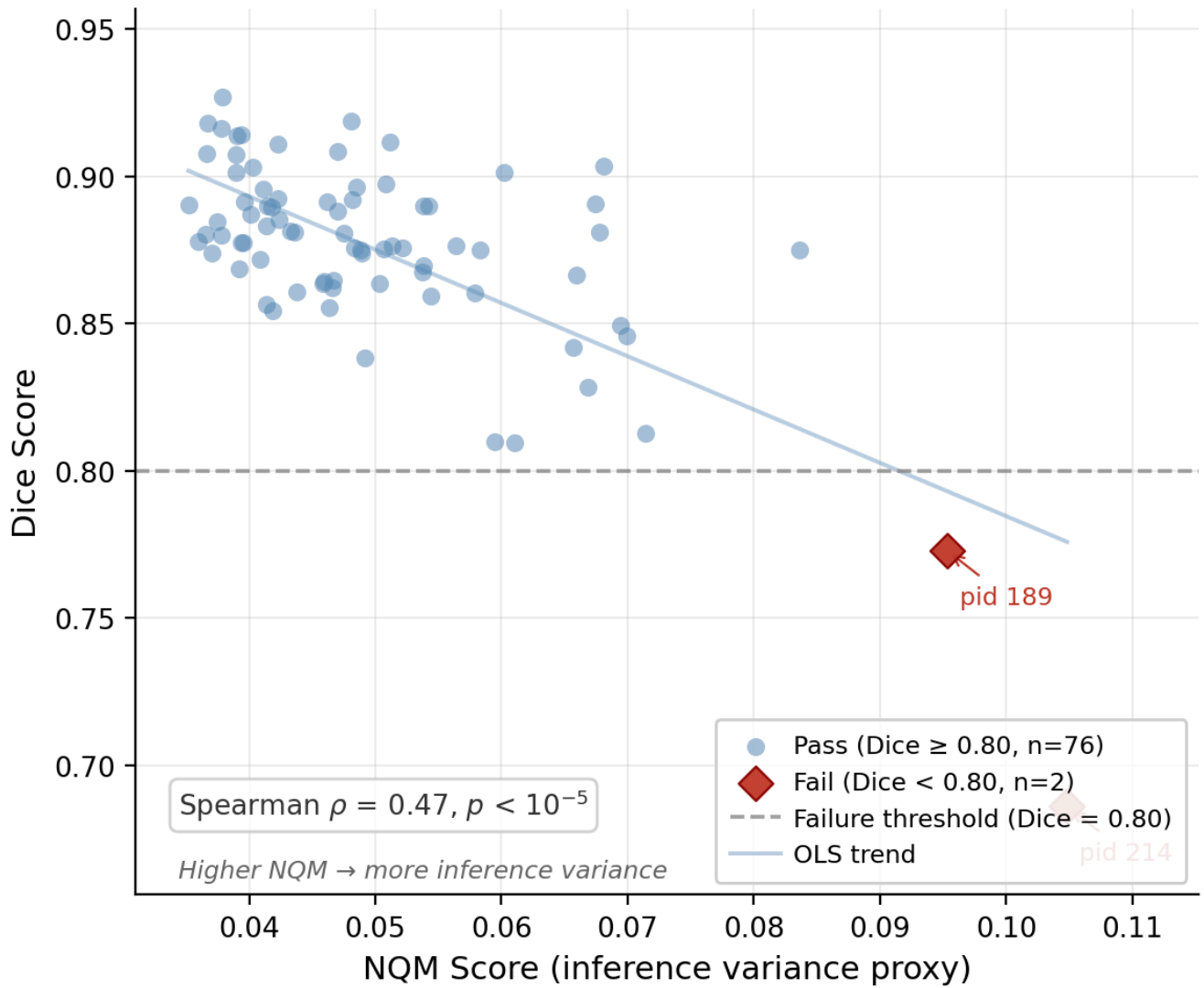


图 7：内建质量指标（V2/R5）。每体积 NQM（推理方差）vs Dice。两个失败案例（红，Dice<0.8）NQM 最高。

效率：两个模型都远在 10 万参数下、显存小，代价是延迟（64 步串行没法跨步并行）。

	海马体	前列腺
参数量	70,016	70,016
峰值显存	待测	121.6 MB
每切片延迟	待测	173.6 ms
计算量	待测	~310 GMACs

海马体的延迟/显存还差一次在冻结 checkpoint 上的测量。

6. 局限（已知的几个弱点）

1. 没跑出一次完整 1000-epoch 的前列腺，所以不能排除某些环境（不同 CUDA/cuDNN、确定性核、或某个没披露的稳定器）能到 0.838。主张是更窄的那条：公开代码在 0/11 次尝试下做不到。
2. 前列腺只有 9 个测试体积，部分跑的 CI 也宽。

3. 官方加载器只留 T2 通道、丢掉 ADC，没改（这是官方行为），但可能跟论文输入不同。
4. 海马体效率延迟/显存待测。
5. 这是复现，只做刻画、没提模型改动。如果之后要做一个能让前列腺训得起来的稳定器（确定性开关、梯度裁剪、多模态输入），那是新工作，应该另开分支，别破坏这里的零偏离记录。

7. 小结

- 海马体复现出来了：0.864 vs 论文 0.886。
- 前列腺在论文规模下没复现出来：0/9 种子（共 0/11 次尝试）到 1000 epoch，最好的部分跑 0.672。原因是官方配置动力学不稳 + cuDNN/atomic 浮点不确定性（固定种子管不住），那次 0.672 是幸存者偏差。
- 可带走的一条经验：跑在稳定边界附近的反复式 NCA 会放大任何不确定性，训练的种子级可重复性不能想当然。

每体积 CSV、冻结测试 ID、完整种子扫描轨迹、种子和 job ID 都留着，每个数字都能查。

附录 A：复现命令

评估是「只评 checkpoint」（不重训）：

```
1 # R1 海马体（本地，仅评估）
2 python code/eval_r1.py \
3     --ckpt checkpoints/r1_hippocampus_official/models/epoch_<E> --seed 42
4
5 # R2 前列腺单次跑（HPC，官方 BackboneNCA）
6 R2_SEED=42 R2_EPOCHS=1000 python code/run_r2_prostate.py
7
8 # R2 9 种子扫描：种子 42..50 每个起一个 job
```

每体积结果：`results/r1_official_{single,pseudo10}.csv`、`results/r2_prostate_{single,pseudo10}.csv`。扫描轨迹：`results/r2_seed_sweep_{traces,dice,summary}.*`。冻结测试 ID：`results/test_ids_r1.txt`。

附录 B：工程踩坑记录

复现一半是数字，一半是坑。按踩到的顺序记一下，省得后面的人再踩。

1. **提速 subclass 发散**。把点火掩码采样挪到 GPU（数学等价）改变了 RNG 流，前列腺发散到 Dice=0（第 4 节）。一个「免费」优化毁了复现。
2. **固定种子不能让训练可复现**。同种子 42 一次收敛一次发散。`torch.manual_seed` 管不住 cuDNN / `atomicAdd` 顺序。要逐 bit 可重复就设 `use_deterministic_algorithms(True)`、`cuda.deterministic=True`、`cuda.benchmark=False`——官方一个没设。
3. **静默发散**。发散不崩——loss 平在 ~5.0、Dice 读 0，但 job 一路跑完。只盯 NaN/OOM 的监控漏掉了，白烧 2.5h GPU。检测器：epoch 10 后 loss>3 且 Dice<0.05 = 已死，立即取消。
4. **早期健康不等于安全**。干净训 60 ~ 121 epoch 的种子仍会一个 epoch 内崩。NCA 用「提前停/提前杀」不安全，得盯到目标 epoch。

附录 C：逐种子扫描数据

9 个种子逐 epoch loss 轨迹在 `results/r2_seed_sweep_traces.csv` (388 行)；逐验证 Dice 在 `r2_seed_sweep_dice.csv`；机读摘要在 `r2_seed_sweep_summary.json`。全局判定：`survived_to_1000ep = 0/9`、`crash_epoch_spread = ep1-ep185`、`best_partial = 0.672 @ ep301`。所有跑用相同官方配置，只有种子变动。